Loops

are a type of control flow statement in programming that enable repetitive execution of a code block. They work alongside conditional statements like if and switch to provide flexibility in algorithm design. While conditional statements dictate whether a specific code runs, loops allow repetition until a condition is met. Loops are broadly categorized into:

1. **Finite Loops**: Repeated a specified number of times.
2. **Conditional Loops**: Continue until a condition is satisfied.

The while Loop

The while loop executes a block of code as long as its condition evaluates to true. It is particularly useful when the number of iterations is not known beforehand.

Syntax:

javascript

Copy code

```
while (condition) {
   // Code block
}
```

Example Without Loop:

To print numbers from 0 to 90 in steps of 10:

javascript

Copy code

```
let n = 0;


console.log(n); // -> 0
n += 10;
console.log(n); // -> 10;
// Repeat this process manually...
```

Example With while Loop:

javascript

Copy code

```
let n = 0;

while (n < 91) {
  console.log(n); // -> 0, 10, 20, ..., 90
  n += 10;
}
```

Key Points:

- The loop condition is evaluated before each iteration.
- The loop stops once the condition evaluates to false.

A Common Use Case:

The while loop shines when the number of iterations is determined dynamically, such as waiting for user input.

javascript

Copy code

```
let isOver = false;
let counter = 1;

while (!isOver) {
  isOver = !confirm(`[${counter++}] Continue the loop?`);
}
```

The do...while Loop

Unlike while, the do...while loop ensures the code block executes **at least once**, as the condition is checked after execution.

Syntax:

javascript

Copy code

```
do {
  // Code block
} while (condition);
```

Example:

javascript

Copy code

```
let isOver;

let counter = 1;


do {

   isOver = !confirm(`[${counter++}] Continue the loop?`);

} while (!isOver);
```

Key Difference: The do...while loop guarantees a minimum of one execution, making it suitable for scenarios where initial execution is necessary before condition evaluation.

The for Loop

The for loop is ideal for cases where the number of iterations is predetermined. It consolidates initialization, condition checking, and increment into one statement.

Syntax:

javascript

Copy code

```
for (initialization; condition; increment) {

   // Code block

}
```

Example:

To print numbers from 0 to 9:

javascript

Copy code

```
for (let i = 0; i < 10; i++) {

   console.log(i);

}
```

Equivalent With while Loop:

javascript

Copy code

```
let i = 0;
while (i < 10) {
   console.log(i);
   i++;
}
```

Key Points:

1. **Initialization**: Executed once at the beginning (e.g., let i = 0).
2. **Condition**: Checked before each iteration (i < 10).
3. **Increment**: Executed at the end of each iteration (i++).

Choosing the Right Loop

Loop Type	Best Use Case
while	When iterations depend on a dynamic condition.
do...while	When at least one execution is necessary.
for	When the number of iterations is predetermined.

Understanding these loop types helps write efficient and readable code, avoiding redundancy and manual repetition.

Loops and Arrays in JavaScript

Dynamic Array Input via Prompt

Create a dynamic list by accepting user inputs until the Cancel button is pressed:

javascript

Copy code

```
let names = [];
let isOver = false;
```

```
while (!isOver) {
   let name = prompt("Enter a name or press Cancel to finish:");
   if (name !== null) {
      names.push(name);
   } else {
      isOver = true;
   }
}

for (let i = 0; i < names.length; i++) {
   console.log(names[i]);
}
```

How It Works:

- The while loop collects names until Cancel is pressed.
- Names are stored in the names array using push.
- A for loop traverses and logs the names.

Traversing Arrays

1. **Forward Traversal**:

javascript

Copy code

```
for (let i = 0; i < values.length; i++) console.log(values[i]);
```

2. **Reverse Traversal**:

javascript

Copy code

```
for (let i = values.length - 1; i >= 0; i--) console.log(values[i]);
```

3. **Step Traversal**:

javascript

Copy code

```
for (let i = 0; i < values.length; i += 2) console.log(values[i]);
```

For...Of Loop

Effortlessly iterate through array elements:

javascript

Copy code

```
let values = [10, 30, 50, 100];
for (let value of values) console.log(value);
```

Example: Filtering Large Cities

javascript

Copy code

```
let cities = [
   { name: "New York", population: 18.65e6 },
   { name: "Delhi", population: 25.87e6 },
   { name: "Tokyo", population: 37.26e6 }
];

for (let city of cities) {
   if (city.population > 20e6) console.log(`${city.name} (${city.population})`);
}
```

For...In Loop

Use to iterate through object keys:

javascript

Copy code

```
let user = { name: "Calvin", age: 66, email: "calvin@example.com" };
for (let key in user) {
   console.log(`${key} -> ${user[key]}`);
}
```

Break and Continue Statements

Break

Exits a loop prematurely:

javascript

Copy code

```
let i = 0;
while (true) {
   console.log(i);
   if (++i >= 5) break;
}
```

Continue

Skips the current iteration:

javascript

Copy code

```
for (let i = 0; i < 10; i++) {
   if (i === 3) continue;
   console.log(i);
}
```

Switch Statements and Break

Without Break (Pass-through)

javascript

Copy code

```
let gate = prompt("Choose gate: a, b, or c");

switch (gate) {
   case "a": alert("Gate A: empty");
   case "b": alert("Gate B: main prize");
   case "c": alert("Gate C: empty");
```

```
    default: alert("Invalid gate.");
}
```

Result: Executes all matching and subsequent cases.

With Break

javascript

Copy code

```
switch (gate) {
    case "a": alert("Gate A: empty"); break;
    case "b": alert("Gate B: main prize"); break;
    case "c": alert("Gate C: empty"); break;
    default: alert("Invalid gate.");
}
```

Fall-through Example

javascript

Copy code

```
switch (gate.toLowerCase()) {
    case "a":
    case "1":
        alert("Gate A: empty");
        break;
    case "b":
    case "2":
        alert("Gate B: main prize");
        break;
    default:
        alert("Invalid gate.");
}
```